



Project Report

on

Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes

Submitted by

Project Members

Soham More (1032222303)

Atharv Prabhune (1032222307)

Swapnaja Magarde (1032222637)

Raviraj Patil (1032222893)

Under the Internal Guidance of

Dr. Sarika Bobde

School of Computer Engineering and
Technology MIT World Peace University,
Kothrud,
Pune 411 038, Maharashtra – India
2025–2026



Dr. Vishwanath Karad
MIT WORLD PEACE
UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

C E R T I F I C A T E

This is to certify that,

Soham More	1032222303
Atharv Prabhune	1032222307
Swapnaja Magarde	1032222637
Raviraj Patil	1032222893

of BTech. (Computer Science & Engineering[Artificial Intelligence and Data Science]) have completed their project titled “Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes” and have submitted this Capstone Project Report towards fulfilment of the requirement for the Degree-Bachelor of Computer Science & Engineering (BTech. - CSE [Artificial intelligence and Data Science]) for the academic year 2025-2026.

Dr. Sarika Bobde

Dr. Balaji M Patil

Project Guide

Program Coordinator

School of CET

School of CET

MIT World Peace University, Pune

MIT World Peace University, Pune

Internal Examiner: _____

External Examiner: _____

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who contributed to the successful completion of this AI-driven project on government scheme data processing, rule extraction, and eligibility evaluation. Finally, we acknowledge the use of multiple open-source libraries and tools such as transformers, scikit-learn, Fast API (planned), and evaluation utilities that enabled efficient automation and analysis. Their ecosystem played a significant role in developing a reliable and reproducible system.

We express our sincere gratitude to our project guide, **Dr. Sarika Bobde**, School of Computer Engineering and Technology, MIT World Peace University, Pune, for her valuable guidance, support, and timely suggestions which helped us successfully complete this project.

We would also like to thank the Program Coordinator, **Dr. Balaji M Patil**, and the faculty members of the Department of Computer Engineering and Technology for their cooperation and support. We are thankful to MIT World Peace University for providing the necessary software resources and academic environment required for this software development project.

Soham More
Final Year B.Tech CSE (AI & DS)
PRN: 1032222303

Atharv Prabhune
Final Year B.Tech CSE (AI & DS)
PRN: 1032222307

Swapnaja Magarde
Final Year B.Tech CSE (AI & DS)
PRN: 1032222637

Raviraj Patil
Final Year B.Tech CSE (AI & DS)
PRN: 1032222893

ABSTRACT

This project develops an AI-powered recommendation system that assists citizens in identifying the most suitable government schemes based on their personal and socio-economic information. The system processes a large dataset of schemes by cleaning, standardizing, and converting unstructured text—such as eligibility criteria and benefits—into structured formats using natural language processing techniques. Eligibility conditions are further transformed into machine-readable JSON rules, enabling accurate, logical evaluation against user-provided profiles.

A hybrid scoring approach is implemented: a rule-based score checks exact eligibility requirements, while a semantic similarity score uses vector embeddings to match user queries with scheme descriptions. These scores are combined to generate ranked and personalized recommendations, along with clear explanations and suggestions to improve eligibility.

By integrating data engineering, vector search, and generative AI, this system enhances accessibility and supports citizens in discovering relevant government welfare schemes efficiently.

List of Figures

4.5.4	Records Found.....	21
4.5.4	Data Entry.....	22
5.1	System Architecture.....	24
5.3	High Level Diagram.....	27
5.4	Low Level Diagram.....	27
5.5	Interaction Diagram.....	28

List of Tables

4.1	Software Tools used	16
4.2	Hardware Required	16
4.3	Requirement- Rationale Table.....	17
4.4	Project Risk Factors.....	18
7.1	Time Complexity.....	40
7.2	Test Cases.....	41

CONTENTS

Abstract	I
List of Figures	II
List of Tables	III

Sr. No.	Name of Sections		Page No.
1	Introduction		1
	1.1	Project statement	1
	1.2	Background	2
	1.3	Objectives	3
	1.4	Aim	3
		1.4.1 Need of project	3
		1.4.2 Implementation Overview	4
		1.4.3 Application of the Project	4
	1.5	Summary	5
2	Literature Review		7
	2.1	Chatbot-Based Government Scheme Assistance	7
	2.2	Hybrid Recommender Approaches	7
	2.3	NLP and Multilingual Systems	8
	2.4	Adoption, Trust, and Public-Sector Chatbots	8
	2.5	Summary of Literature Gaps	9
3	Problem Statement		10
	3.1	Scope of the project	11
	3.2	Project Limitations	12
		3.2.1 Functional Limitations	12
		3.2.2 Technical Limitations	12
		3.2.3 User-Related Limitations	13
	3.3	Project Assumptions	13
		3.3.1 User Profile Schema Assumptions	13
		3.3.2 Technical Assumptions	14

		3.3.3	Environment & Dependency Assumptions	14
4	Project Requirements			15
	4.1	Software requirements		16
	4.2	Hardware requirements		16
	4.3	Requirements Rationale		17
	4.4	Risk Management		18
	4.5	Functional Specifications		19
		4.5.1	External interfaces required	19
		4.5.2	Internal interfaces required	19
		4.5.3	Communication interfaces	21
		4.5.4	Graphical User Interface (GUI)	21
5	System Analysis Proposed Architecture			23
	5.1	System Architecture		24
	5.2	Modules of the Project		24
	5.3	High Level diagram		27
	5.4	Low level Design		27
	5.5	UML Diagram		28
6	Implementation			29
	6.1	Methodology		30
	6.2	Algorithm		35
	6.3	Data Set Description		36
7	Performance Evaluation and Testing			39
	7.1	Time Complexity		40
	7.2	Test Case		41
8	Deployment Strategies			42
	8.1	Local development deployment		43
	8.2	Production deployment		43
9	Result and Analysis			44
	9.1	Rule Evaluation Results		45
	9.2	Retrieval and Ranking Accuracy		45

	9.3	Analysis of Ranking Behaviour	46
	9.4	Error and Failure Analysis	46
	Application		47
	Future prospects of the project		49
	Conclusion		50
	References		51
	Plagiarism Report		

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
FAISS	Facebook AI Similarity Search
GUI	Graphical User Interface
API	Application Programming Interface
IP	Inner Product

CHAPTER - 1

INTRODUCTION

1. INTRODUCTION

Accessing suitable government welfare schemes is often challenging for citizens due to the vast number of programs, complex eligibility criteria, and fragmented information across multiple platforms. This results in low awareness, mismatched applications, and limited utilization of available benefits. To address these challenges, the AI-Based Government Scheme Recommendation System leverages modern artificial intelligence techniques to simplify and optimize the scheme-selection process for users.

The system integrates Large Language Models (LLMs), semantic similarity search, and a rule-based eligibility engine to analyse user inputs and generate accurate, personalized recommendations. By processing thousands of government schemes and automatically structuring their eligibility criteria, the system enables efficient matching between user profiles and scheme requirements. Through a hybrid architecture combining NLP-driven semantic analysis with deterministic rule evaluation, the model enhances reliability, transparency, and decision accuracy.

This project aims to improve accessibility to welfare programs, reduce manual search effort, and provide citizens with a streamlined, intelligent interface for discovering relevant government schemes. The system demonstrates how AI can be effectively applied to public-service domains to improve efficiency, user experience, and overall scheme reach.

1.1 PROBLEM STATEMENT

Identifying the most relevant government welfare schemes is difficult due to unstructured policy documents, inconsistent eligibility descriptions, and the lack of an intelligent recommendation system. Existing portals rely mainly on keyword matching and do not evaluate a citizen's actual eligibility, resulting in low-accuracy and non-personalized results.

This project aims to address this gap by developing an AI-based hybrid recommendation system that processes 3,400+ schemes, extracts structured eligibility rules using LLM-driven text processing, and evaluates them through a deterministic rule-engine. A semantic similarity model is integrated to capture contextual relevance between user queries and scheme descriptions. Finally, a hybrid scoring mechanism combines rule-match, semantic relevance, and scheme

freshness to rank schemes effectively.

The core problem is to design a scalable system capable of transforming unstructured policy text into machine-readable rules and accurately recommending welfare schemes based on a citizen's profile and needs using a combination of AI, embeddings, and logical reasoning.

1.2 BACKGROUND

Accessing government welfare schemes in India is often hindered by fragmented information, inconsistent eligibility descriptions, and the absence of a unified retrieval system. Existing studies emphasize that manual navigation of large policy datasets significantly affects citizen accessibility, especially for marginalized groups [1]. Prior research in public-policy informatics also highlights that eligibility criteria are written in unstructured, ambiguous formats, making automated interpretation difficult without advanced NLP models [2].

Recent advances in natural language processing (NLP), large language models (LLMs), and knowledge-graph methods provide new possibilities for transforming these unstructured government documents into structured, machine-processable formats [3]. Embedding-based semantic similarity has been shown to improve retrieval quality in large text corpora, enabling more accurate matching between user queries and relevant policy interventions [4]. Additionally, rule-based reasoning systems have demonstrated strong performance in high-precision eligibility verification tasks when combined with machine-learning models [5].

Studies on AI-assisted public-service delivery suggest that hybrid models (LLM + symbolic systems) outperform pure machine-learning systems by providing transparency, interpretability, and error-resilience in decision workflows [6]. This aligns with research in human-centered AI, which recommends combining semantic retrieval with deterministic rule engines to ensure both accuracy and fairness in citizen-facing applications [7].

Informed by these findings, this project integrates three core components:

1. **Semantic Matching** using vector embeddings to identify relevant schemes.
2. **Structured Rule Extraction** using LLM-based text-to-rules parsing.
3. **Rule-Based Eligibility Evaluation** to deliver precise, personalized recommendations.

By synthesizing these methods, the system addresses long-standing challenges in scheme discovery, simplifies the user experience, and advances the application of AI in public governance.

1.3 OBJECTIVES

1. To design and implement a hybrid AI driven recommendation engine that combines semantic embeddings, rule-based eligibility matching, and personalized scoring for accurate government scheme retrieval.
2. To develop a structured eligibility extraction pipeline that converts unstructured scheme descriptions into machine readable rule sets using LLM based parsing and validation.
3. To build a user profile-aware retrieval system that dynamically adjusts recommendation scores based on demographic, occupational, financial, and categorical attributes.
4. To evaluate system performance using quantitative metrics such as semantic relevance accuracy, rule engine precision, personalized match improvement, and ranking consistency across realistic test cases.
5. To create an end-to-end deployable system architecture integrating data ingestion, preprocessing, embeddings generation, FAISS based vector search, rule evaluation, and scoring into a unified pipeline.

1.4 AIM

1.4.1 Need of the Project

Accessing government welfare schemes in India is a highly fragmented and complex process due to the vast number of schemes, frequent policy updates, varying eligibility rules, and lack of a unified knowledge source. Citizens—especially those from rural, low-literacy, and economically weaker backgrounds—often struggle to identify which schemes they qualify for. Existing portals provide static information but fail to offer personalized, context-aware recommendations. Moreover, eligibility criteria are written in unstructured natural language, making manual interpretation time-consuming and error-prone.

Thus, there is a strong need for an AI-based intelligent recommendation system capable of processing large-scale scheme data, understanding user intent, extracting eligibility rules automatically, and generating accurate, personalized results. Such a system enhances

accessibility, reduces information gaps, and empowers citizens to make informed welfare decisions.

1.4.2 Implementation Overview

The project implements a hybrid AI architecture that integrates Natural Language Processing (NLP), semantic embeddings, rule-based engines, and vector search technologies. First, unstructured scheme descriptions are processed using an LLM to extract structured eligibility rules. These are stored as machine-readable conditions. User queries are encoded using MiniLM sentence embeddings, and FAISS is employed to retrieve semantically relevant schemes at high speed.

A deterministic eligibility rule engine then evaluates user inputs such as age, occupation, income, state, and social category against the structured rules to calculate a rule-match score. A hybrid ranking algorithm combines rule-score, semantic-score, and freshness-score to ensure both relevance and correctness. Finally, a clean, intuitive UI is used to gather user data and display the results with clear explanations, ensuring transparency and usability.

This multi-stage pipeline ensures accuracy, explainability, scalability, and real-time performance.

1.4.3 Application of the Project

The proposed system can be deployed in multiple real-world environments. Government websites and helpline portals can use it to provide citizens with personalized recommendations instead of generic information. It can be integrated into mobile applications, enabling easy access for rural and digitally inexperienced users. Additionally, Common Service Centres (CSCs) and NGOs working on welfare distribution can adopt the system to assist beneficiaries during scheme enrollment.

The system can also be used by researchers and policymakers to analyze eligibility patterns, identify underserved demographic groups, and evaluate the accessibility of welfare programmes. Its modular AI architecture makes it suitable for future integration with official APIs such as MyScheme, Digital India services, and state-specific governance platforms.

1.5 SUMMARY

The project addresses the difficulty citizens face in identifying suitable welfare schemes due to unstructured eligibility rules and scattered information. It develops a hybrid AI-based recommendation system that integrates semantic embeddings, LLM-driven rule extraction, and a deterministic rule-evaluation engine. The system processes 3,400+ schemes, structures their eligibility, and ranks results using rule-match and semantic relevance scores. Its scope includes data preprocessing, rule extraction, vector search, scoring, UI design, and performance evaluation. Overall, the project aims to deliver a scalable, accurate, and user-friendly platform for personalized scheme discovery.

CHAPTER-2

LITERATURE REVIEW

2. LITERATURE REVIEW

The development of AI-driven welfare-scheme recommendation systems has been explored across chatbot research, hybrid recommender models, and multilingual NLP frameworks. Existing studies provide useful foundations but often lack scalability, structured eligibility processing, and deep personalization. This literature review summarizes prior work and highlights the gaps that motivate our hybrid AI approach.

2.1 CHATBOT-BASED SCHEME ASSISTANCE

Early works such as SchemeBot [4], Scheme Setu [5], and Python-based chatbots [7] focus on providing basic conversational access to government schemes using simple NLP techniques and static databases. Seq2Seq-based approaches [6] improve natural-language response fluency and allow more flexible interactions. These tools demonstrate the initial progress toward digitizing scheme discovery and enabling automated citizen interactions.

However, these systems exhibit several limitations: they rely heavily on keyword matching, lack semantic retrieval, cannot process large scheme datasets, and provide no systematic eligibility verification. Seq2Seq models generate responses but cannot enforce correctness or prevent hallucination. Most solutions support only limited languages, are not scalable, and require manual updates of scheme data. None use LLM-based rule extraction or hybrid scoring—major gaps addressed by the current project.

2.2 HYBRID RECOMMENDER APPROACHES

Mandalageri et al. [1] introduce a hybrid recommender combining content-based filtering, behavioral filtering, and rule-based logic. Their work highlights how integrating ML with deterministic rule checks can improve service allocation and personalization. It also establishes that user profiles and past behavior can enhance recommendation quality.

Despite these strengths, significant gaps remain: their system operates on small datasets, does not parse unstructured eligibility text, and cannot scale to thousands of government schemes. It also lacks semantic embeddings, clause-level reasoning, and explainability. Importantly, the model does not integrate with real-time data sources like government portals. These gaps motivate our hybrid approach using LLM-based rule extraction, FAISS embeddings, and explainable scoring.

2.3 NLP AND MULTILINGUAL SYSTEMS

NLP-driven systems such as the Smart Scheme Navigation System [2] utilize BERT, multilingual text processing, and modern NLP pipelines to assist Tamil Nadu citizens. Similarly, IndicTrans2 [9] enables high-quality translation across all 22 Scheduled Indian languages, providing a strong backbone for multilingual government-service delivery.

However, these models fall short in key areas: they do not incorporate rule-checking mechanisms, cannot understand complex eligibility constraints, and struggle with domain-specific government terminology. They also do not automatically update scheme data or provide hybrid retrieval capabilities. Therefore, while they offer strong linguistic foundations, they lack the logical reasoning and large-scale coverage required for accurate scheme recommendation—capabilities addressed in our system.

2.4 ADOPTION, TRUST, AND PUBLIC-SECTOR CHATBOTS

Studies such as Rathnayake et al. [3] and Papadopoulos et al. [8] focus on usability, trust, ethics, and design quality in public-sector chatbots. These works suggest that citizens evaluate government chatbots not only on accuracy but also on reliability, transparency, and user experience. They emphasize the need for systems that explain decisions, protect user data, and maintain consistent performance.

Yet, these works also reveal gaps: they focus on perception and adoption rather than technical architecture, and do not address eligibility logic, retrieval algorithms, or hybrid ranking. They lack practical models for large-scale scheme recommendation and do not propose methods for structured rule evaluation. This highlights the need for systems like ours that combine technical accuracy with user-friendly, trust-building interfaces.

2.5 SUMMARY OF LITERATURE GAPS

Across all reviewed literature, the following persistent gaps exist:

1. No scalable solution for structured eligibility extraction from unstructured text.
2. Lack of hybrid semantic + rule-based ranking.
3. Minimal user-profile personalization.
4. Limited multilingual retrieval capability.
5. Little focus on explainability and clause-level transparency.
6. Absence of large-scale scheme datasets (3,000+).

CHAPTER - 3

PROBLEM STATEMENT

3.1 SCOPE OF THE PROJECT

Accessing government welfare schemes in India is often hindered by fragmented information, inconsistent eligibility descriptions, and the absence of a unified retrieval system. Existing studies emphasize that manual navigation of large policy datasets significantly affects citizen accessibility, especially for marginalized groups [1]. Prior research in public-policy informatics also highlights that eligibility criteria are written in unstructured, ambiguous formats, making automated interpretation difficult without advanced NLP models [2].

Recent advances in natural language processing (NLP), large language models (LLMs), and knowledge-graph methods provide new possibilities for transforming these unstructured government documents into structured, machine-processable formats [3]. Embedding-based semantic similarity has been shown to improve retrieval quality in large text corpora, enabling more accurate matching between user queries and relevant policy interventions [4]. Additionally, rule-based reasoning systems have demonstrated strong performance in high-precision eligibility verification tasks when combined with machine-learning models [5].

Studies on AI-assisted public-service delivery suggest that hybrid models (LLM + symbolic systems) outperform pure machine-learning systems by providing transparency, interpretability, and error-resilience in decision workflows [6]. This aligns with research in human-centered AI, which recommends combining semantic retrieval with deterministic rule engines to ensure both accuracy and fairness in citizen-facing applications [7].

Informed by these findings, this project integrates three core components:

1. Semantic Matching using vector embeddings to identify relevant schemes.
2. Structured Rule Extraction using LLM-based text-to-rules parsing.
3. Rule-Based Eligibility Evaluation to deliver precise, personalized recommendations.

By synthesizing these methods, the system addresses long-standing challenges in scheme discovery, simplifies the user experience, and advances the application of AI in public governance.

3.2 PROJECT LIMITATIONS

3.2.1 Functional Limitations

1. Dependency on Semantic Retrieval

The system ranks only those schemes returned by semantic search; if retrieval fails, eligible schemes may be missed.

2. Limited Coverage of Complex Eligibility Rules

Regex-based rule extraction cannot fully interpret nuanced, legal, or multi-layered eligibility conditions.

3. Restricted Freshness Logic

Freshness score uses a simple 1-year threshold; does not account for policy amendments, source reliability, or state-specific update cycles.

4. Incomplete Normalization Coverage

State names, caste labels, occupations, and uncommon aliases may not be perfectly normalized.

5. No Real-Time Scheme Updates

System assumes updated datasets; does not pull live data from government portals unless manually refreshed.

3.2.2 Technical Limitations

1. Dependency & Version Inconsistencies

Multiple requirements files; missing critical packages (faiss-cpu, sentence-transformers, pydantic).

2. Indexing and ID Mapping Mismatch

FAISS index files and scheme ID maps have inconsistent naming conventions → can

break semantic retrieval.

3. Fragile Rule Extraction

Regex-heavy extraction can misinterpret unit conversions, numeric ranges, or ambiguous phrasing.

4. Unit Handling Issues

Land area conversions (acres ↔ hectares) may not always align between user profile and extracted rules.

5. Brute-Force FAISS Index

FAISS IndexFlatIP is not approximate; scales linearly → slow for very large datasets.

3.2.3 User Limitations

1. Incorrect or Missing User Input

Eligibility evaluation depends heavily on accuracy of age, income, occupation, land area, etc.

2. Low Digital Literacy

Users unfamiliar with scheme categories or demographic fields may provide incomplete profiles.

3.3 PROJECT ASSUMPTIONS

3.3.1 User Profile Schema Assumptions

1. The recommendation pipeline assumes a canonical user profile model with standardized fields such as state, age, gender, category, income_annual, occupation, farmer status, land_area, etc.
2. Extra unsupported fields are safely ignored; input values are assumed to be normalized before processing
3. Raw user inputs may contain varied naming formats (e.g., “annual_income”), which are

mapped to canonical keys (e.g., income_annual).

4. Values for categorical fields (state, gender, caste, education, booleans) are normalized using deterministic mappings.

3.3.2 Technical Assumptions

1. Eligibility extraction is handled primarily using regex-based heuristics, with an optional LLM fallback.
2. Rules are assumed to be structured into required and optional clauses with fields, operators, values, and confidence.
3. “Other” clauses may be heuristically mapped to known fields such as state, occupation, income_annual, age, etc.
4. A field-mapping file is assumed to exist to align rule fields with UserProfile fields.
5. Scheme data is automatically cleaned, translated (if required), and synthesized to ensure all fields contain usable text.
6. Embedding documents are capped at ~4000 characters.

3.3.3 Environment & Dependency Assumptions

1. All required Python libraries are available (pandas, pyarrow, bs4, pdfplumber, langdetect, googletrans, pydantic, sentence-transformers, faiss-cpu, numpy).
2. Consistent dependency versions are assumed.

CHAPTER - 4

PROJECT REQUIREMENT

4.1 SOFTWARE REQUIREMENTS

Category	Tools / Libraries
Programming Language	Python 3.10+
Core Libraries	pandas, numpy, pyarrow, pydantic
NLP/ML Libraries	sentence-transformers, faiss-cpu, sklearn
LLM / AI Components	google-generativeai or OpenAI API (optional for fallback rule extraction)
Text Processing	pdfplumber, BeautifulSoup4, langdetect, googletrans
Backend / Utility	requests, python-dotenv
Version Control	Git / GitHub

Table4.1: Software tools used

4.2 HARDWARE REQUIREMENTS

Component	Requirement
CPU Machine	Minimum 8 GB RAM for embedding generation
GPU (optional)	Recommended for faster embedding generation (NVIDIA 4–8 GB VRAM)
Storage	At least 2–5 GB for cleaned datasets, embeddings, and FAISS index
Internet	Required for translation, LLM services, dataset updates

Table4.2: Hardware required

4.3 REQUIREMENTS RATIONALE

Requirement	Rationale
The system must extract eligibility rules from unstructured text.	Eligibility details are often embedded in PDFs or long paragraphs; structured rules enable accurate evaluation.
The system must normalize user profile inputs.	Users may enter non-standard spellings, formats, or aliases; normalization ensures consistent rule evaluation.
The pipeline must generate semantic embeddings for all schemes.	Embeddings allow high-quality similarity search between user queries and scheme descriptions.
The system must maintain a FAISS vector index.	Required for fast, scalable retrieval across thousands of scheme documents.
The ranking engine must combine rule-matching and semantic similarity.	Hybrid scoring ensures both relevance (semantic) and correctness (rule-based eligibility).
The UI must collect all essential user attributes.	Complete profiles improve accuracy of rule evaluation and personalized recommendations.
The system must tolerate missing or unknown fields.	Government datasets are inconsistent; robustness is necessary.
The system must generate explainable outputs.	Users need clarity on <i>why</i> a scheme is recommended or rejected (trust & transparency).
The architecture must support adding new schemes easily.	Government schemes change frequently; maintainability is crucial.
The pipeline must be modular.	Enables reusability, debugging ease, and independent updates to extraction, embedding, and ranking.

Table4.3: Requirements–Rationale Table

4.4 RISK MANAGEMENT

Risk Factor	Description	Impact Level	Likelihood	Overall Risk
Data Quality Issues	Scheme data may contain missing fields, outdated information, or inconsistent formatting affecting eligibility extraction and ranking.	High	Medium	High
Rule Extraction Errors	Regex-based and heuristic rule extraction may misinterpret eligibility criteria, reducing accuracy of recommendations.	High	Medium	High
Dependency & Library Mismatch	Missing or outdated Python libraries (e.g., FAISS, pydantic) may cause pipeline failures during setup or execution.	Medium	Medium	Medium
Scalability Limitations	FAISS IndexFlatIP (brute force) may become slow as dataset grows, impacting retrieval speed.	Medium	High	High
User Profile Inconsistency	Users may provide incomplete, ambiguous, or incorrectly formatted profile fields, affecting normalization and ranking.	Medium	Medium	Medium

Table4.4: Project Risk factors

4.5 FUNCTIONAL SPECIFICATIONS

4.5.1 External Interfaces Required

1. Government Scheme Data Interface

- Method: `fetch_scheme_data()`
- Action: Pulls raw scheme documents, CSVs, and updates from external portals/APIs.
- Interaction: Connects to public datasets, HTML pages, or APIs (if available) to fetch the latest scheme information.

2. Translation / Language Processing Interface

- Method: `translate_text(text, target_lang)`
- Action: Translates non-English scheme content to English for uniform processing.
- Interaction: Uses external translation engines (e.g., Google Translate API).

3. LLM/Embedding Model Interface

- Method: `generate_embedding(text)`
- Action: Sends cleaned text to the sentence-transformer model to produce embedding vectors.
- Interaction: Calls local/hosted embedding model services.

4.5.2 Internal Interface Required

1. Scheme Cleaning → Rule Extraction

- Method: `process_schemes.py` → `extract_eligibility_rules.py`
- Action: Passes cleaned text
- Interaction: Structured rules generated from cleaned eligibility fields

2. Rule Extraction → Rule Mapping

- Method: `extract_eligibility_rules.py` → `build_field_mapping.py`
- Action: Maps raw rule fields to profile fields
- Interaction: Ensures compatibility with UserProfile model

3. Rule Mapping → Rule Evaluator

- Method: `build_field_mapping.py` → `rule_evaluator.py`
- Action: Provides normalized clauses
- Interaction: Evaluator checks rules using operators (=, <, between, etc.)

4. Embedding Docs → Embeddings

- Method:
`build_embedding_docs.py` → `compute_scheme_embeddings.py`
- Action: Embedding-ready documents
- Interaction: Embeddings computed using ST model

5. Embeddings → FAISS Index

- Method: `compute_scheme_embeddings.py` → `build_faiss_index.py`
- Action: Stores vector representations
- Interaction: Creates inner-product FAISS index

6. Query Generator → Semantic Retrieval

- Method: `semantic_retrieval.py`
- Action: Constructs & embeds query
- Interaction: Query is compared with FAISS vectors for top match retrieval

4.5.3 Communication Interface Required

1. File-Based Communication:

- Uses .parquet, .npy, .bin, .json files as intermediates between pipeline steps.
- Ensures standardized data formats for scheme lists, embeddings, IDs, rules.

2. Function-Call Communication:

Each pipeline step calls the next in sequence (e.g., cleaning → rule extraction → embeddings → FAISS index).

3. Model Communication:

Components share encoded vectors through FAISS and embedding modules.

4. Query Communication:

User query + selected profile fields → embedding model → FAISS → ranking.

4.5.4 Graphical User Interface (GUI)

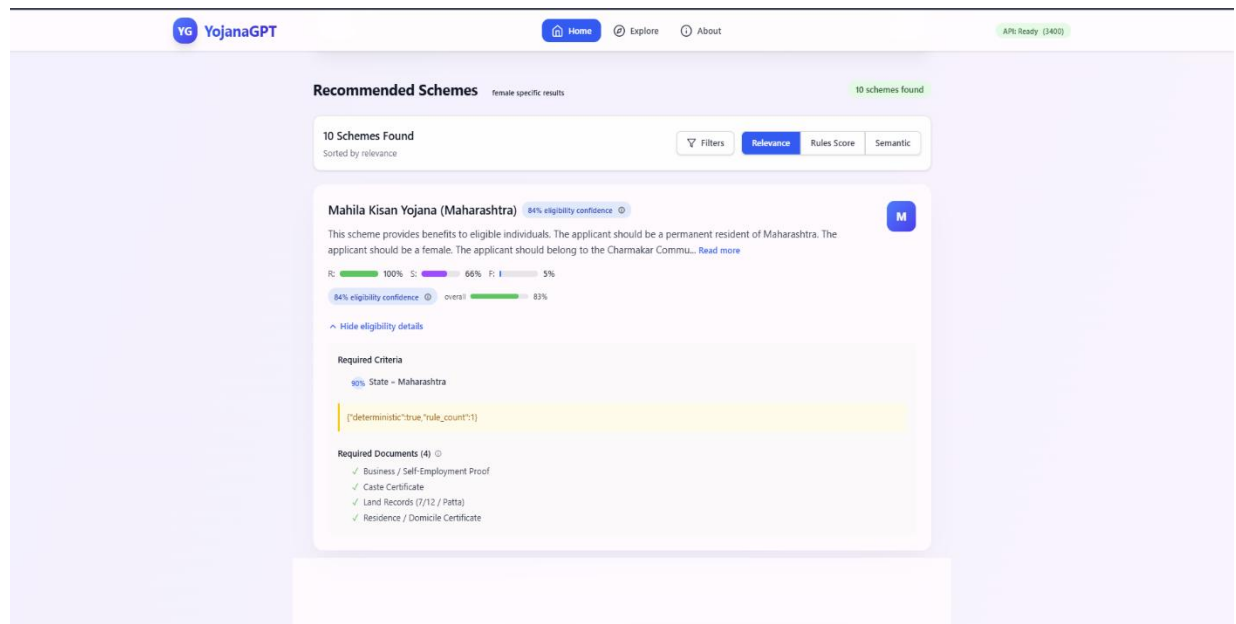


Figure 4.5.4: Records found

YojanaGPT

Government Scheme Recommender

Find schemes tailored to your profile and query with transparent eligibility details

Find Your Perfect Scheme

Fill in your details to discover government schemes tailored for you

Education

Agriculture

Healthcare

Scholarship

Loan

Housing

Startup

State *

📍 Maharashtra

▼

Gender *

👤 Female

▼

Monthly Income (₹) *

\$ 5000

Occupation *

🏠 Homemaker

▼

Show Advanced Filters ▼

↺ Reset Form

🔍 Search Schemes

Figure 4.5.4: Data Entry

22

CHAPTER - 5

SYSTEM ANALYSIS PROPOSED ARCHITECTURE

5.1 SYSTEM ARCHITECTURE

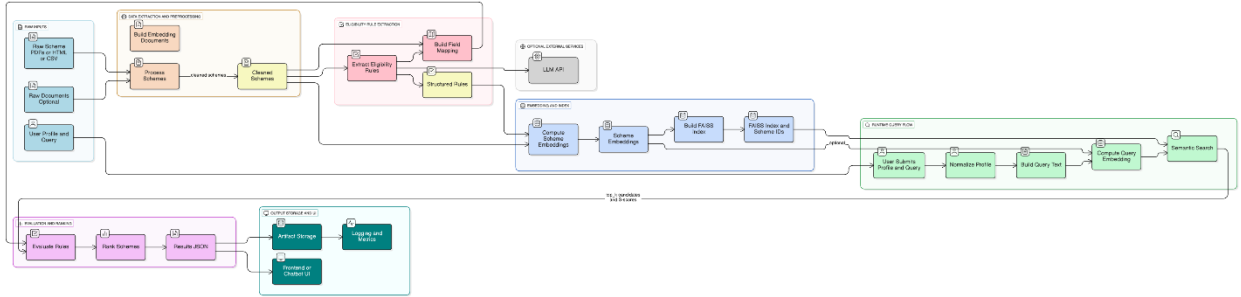


Figure 5.1: System Architecture

5.2 MODULES OF PROJECT

1. Data Collection & Preprocessing Module

Objective: Collect raw scheme documents (PDF/HTML/CSV) and clean them.

Responsibilities:

- Extract text from PDFs/HTML.
- Remove noise, symbols, duplicates.
- Translate non-English text to English if required.
- Store cleaned text in structured format (Parquet).

2. Eligibility Rule Extraction Module

Objective: Convert scheme documents into structured eligibility rules.

Responsibilities:

- Use regex + heuristics to extract age, income, occupation, category, land area, etc.
- Break rules into *required* and *optional* clauses.
- Assign confidence scores.
- Map rule fields to user profile fields.

3. Embedding Generation Module

Objective: Create vector embeddings (numerical representation) of scheme documents.

Responsibilities:

- Build scheme embedding documents (title + summary + benefits + eligibility).
- Encode using Sentence-Transformer (paraphrase-mpnet-base-v2).
- Generate and store 768-dim embeddings.

4. FAISS Indexing & Retrieval Module

Objective: Enable fast and accurate semantic search.

Responsibilities:

- Build a FAISS IndexFlatIP for cosine similarity search.
- Load embeddings and IDs.
- Retrieve top-K similar schemes based on query + profile.

5. User Profile Normalization Module

Objective: Convert user inputs into a consistent internal format.

Responsibilities:

- Standardize fields like state, gender, category, occupation.
- Normalize income formats (annual/monthly).
- Convert land units to common units (acre/hectare).
- Handle missing/extra fields gracefully.

6. Rule Evaluation & Ranking Module

Objective: Filter and score schemes based on eligibility and semantic relevance.

Responsibilities:

- Evaluate required/optional rules for each scheme.
- Calculate rule-score (R), semantic-score (S) and freshness penalty (F).
- Compute final ranking score:

$$Score = clamp(w_r R + w_s S - w_f F)$$

- Return top recommendations.

7. Backend API Module

Objective: Provide services for frontend to communicate with the model.

Responsibilities:

- FastAPI endpoints (e.g., /recommend).
- Load FAISS, models, scheme data.
- Validate user input.
- Return final recommended schemes as JSON.

8. Frontend UI Module

Objective: Provide a simple user interface for interaction.

Responsibilities:

- User profile form (state, income, occupation, etc.).
- Query input box.
- Display ranked schemes with eligibility explanations.

9. Logging & Debugging Module

Objective: Help developers track system performance and errors.

Responsibilities:

- Print diagnostic info during extraction and ranking.
- Log unmatched rules, unknown fields, errors.
- Provide test utilities.

10. Testing & Validation Module

Objective: Ensure system correctness and stability.

Responsibilities:

- Unit tests for rule evaluator.
- Scenario tests for ranking.
- Extraction tests.
- pytest-based automation.

5.3 HIGH LEVEL DIAGRAM

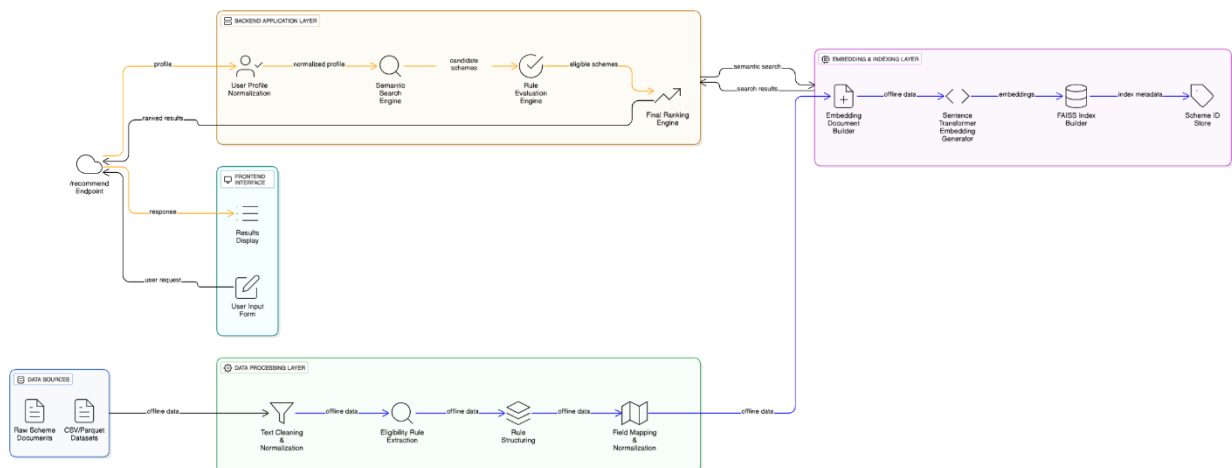


Figure 5.3: High level diagram

5.4 LOW LEVEL DIAGRAM

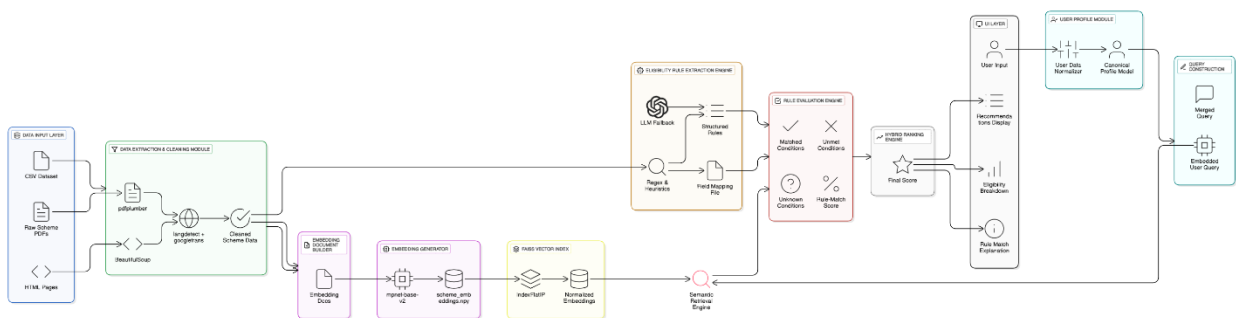


Figure 5.4: Low Level Diagram

5.5 INTERACTION DIAGRAM (UML DIAGRAM)

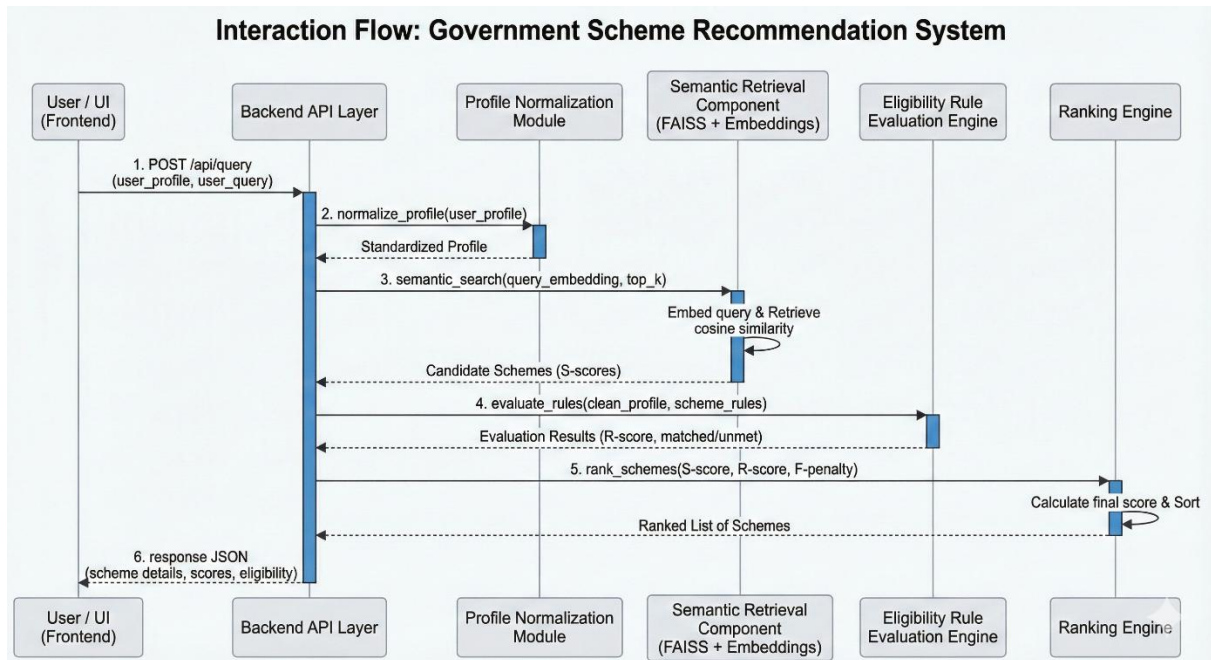


Figure 5.5: Interaction Diagram

CHAPTER - 6

IMPLEMENTATION

6.1 METHODOLOGY

The implementation of this project follows a structured and systematic methodology designed to convert raw government scheme data into accurate, personalized recommendations for users. The process integrates rule-based eligibility extraction, semantic retrieval using embeddings, and a hybrid ranking mechanism to ensure that each user receives relevant and verified scheme suggestions. By combining data preprocessing, machine learning models, and a well-defined evaluation pipeline, the system ensures consistency, transparency, and scalability. This section outlines the methodology step-by-step—illustrated with a practical test-case example—and details the core algorithmic flow that drives the recommendation engine.

1. Test case

User profile (raw):

- State: Maharashtra
- DOB / Age: 1990-06-15 → Age = 35
- Occupation: Farmer
- Annual family income: ₹1,50,000
- Land area: 2 hectares
- Category: OBC
- Free-text query: "subsidy for drip irrigation"

We normalize this into a canonical profile (fields standardized):

```
{
  "state": "Maharashtra",
  "age": 35,
  "occupation": "farmer",
  "income_annual": 150000,
  "land_area_hectares": 2.0,
  "category": "OBC",
  "query_text": "subsidy for drip irrigation"
}
```


2. Preprocessing & Rule extraction (how rules are produced)

Assume we have a scheme with this raw eligibility text:

Scheme A (Irrigation Support X)

"Small and marginal farmers of Maharashtra having cultivable land up to 5 hectares are eligible. Annual family income must be below ₹2,00,000. Priority to OBC/SC/ST applicants. Age must be 18–60 years."

Rule extraction (**regex + heuristics**) picks patterns like:

- `land_area <= 5 hectares` → clause: {field: `land_area_hectares`, `<=`, value: 5, type: required}
- `income < 200000` → {income_annual, `<`, 200000, required}
- `category in [OBC, SC, ST]` → {category, in, [OBC,SC,ST], optional/priority}
- `age between 18 and 60` → {age, between, [18,60], required}

(If a sentence was ambiguous, an LLM fallback would produce a structured clause in the same format.)

3. Build embedding docs & semantic retrieval (S score)

We compute embeddings for each scheme doc and for the user query ("subsidy for drip irrigation" plus short profile fields). FAISS gives semantic similarities (cosine-like values normalized to [0,1]).

Assume FAISS returns top 3 candidates with semantic scores S:

- Scheme A: S = 0.82
- Scheme B: S = 0.70
- Scheme C: S = 0.45

4. Rule evaluation per candidate → compute R score

We evaluate each scheme's structured clauses against the canonical user profile. Each clause yields matched / unmet / unknown. We weight required clauses higher than optional ones.

Define simple weights:

- Required clause weight = 1.0
- Optional clause weight = 0.5

$$R = \frac{\sum(\text{clause weight} \times \text{clause_confidence} \times I)}{\sum(\text{clause_weight})}$$

I = Index matched

(clause_confidence $\in$ (0,1] — assume heuristics give high confidence for regex rules, e.g. 0.9)

Example:

Scheme A rules (from above)

Clauses:

1. land_area_hectares ≤ 5 — required — confidence 0.9
2. income_annual < 200000 — required — confidence 0.9
3. category in [OBC,SC,ST] — optional — confidence 0.8
4. age between 18–60 — required — confidence 0.9

Evaluate against user:

1. user land = 2 $\leq 5 \rightarrow$ matched
2. income 150000 $< 200000 \rightarrow$ matched
3. category OBC $\rightarrow$ matched
4. age 35 in 18–60 $\rightarrow$ matched

Compute denominator (total weight):

$$= \text{required}(1.0) + \text{required}(1.0) + \text{optional}(0.5) + \text{required}(1.0) = 3.5$$

Compute numerator:

$$\begin{aligned} &= (1.0 * 0.9 * 1) + (1.0 * 0.9 * 1) + (0.5 * 0.8 * 1) + (1.0 * 0.9 * 1) \\ &= 0.9 + 0.9 + 0.4 + 0.9 = 3.1 \end{aligned}$$

So

$$R_a = \frac{3.1}{3.5} = 0.8857 \approx 0.886$$

Suppose Scheme B has rules:

- only for state = Karnataka (required) → unmet (user Maharashtra)
- income < 3,00,000 (required) → matched
- age 21–50 (required) → matched

Weights sum = 3.0 (three required)

Numerator = (0 + 0.9 + 0.9) = 1.8 →

$$R_b = \frac{1.8}{3} = 0.60$$

Scheme C

Scheme C is a national scheme vaguely related; rules missing or unclear → many unknowns:

eligibility unclear → total weight small (say 1 required), matched = 0 →

$$R_c = 0.0$$

5. Freshness penalty F

Compute freshness penalty $F \in [0, 0.1]$ from `last_updated` date.

Example:

- Scheme A: `last_updated` 3 months ago → $F_A = 0.01$ (very fresh)
- Scheme B: 18 months ago → $F_B = 0.08$ (older)
- Scheme C: 3 years ago → $F_C = 0.10$ (max penalty)

6. Hybrid ranking formula & final score

We choose weights (tunable):

- $w_r = 0.6$ (rule importance)
- $w_s = 0.4$ (semantic importance)
- $w_f = 0.1$ (freshness factor multiplied directly subtractively)

Final score:

$$Final = clamp(w_r \cdot R + w_s \cdot S - w_f \cdot F, 0, 1)$$

Plug values:

- Scheme A: $R=0.886$, $S=0.82$, $F=0.01$
 $\text{Final_A} = 0.6 \times 0.886 + 0.4 \times 0.82 - 0.1 \times 0.01$
 $= 0.5316 + 0.328 - 0.001 = 0.8586$
- Scheme B: $R=0.60$, $S=0.70$, $F=0.08$
 $\text{Final_B} = 0.6 \times 0.60 + 0.4 \times 0.70 - 0.1 \times 0.08$
 $= 0.36 + 0.28 - 0.008 = 0.632$
- Scheme C: $R=0.0$, $S=0.45$, $F=0.10$
 $\text{Final_C} = 0.6 \times 0.0 + 0.4 \times 0.45 - 0.1 \times 0.10$
 $= 0 + 0.18 - 0.01 = 0.17$

Sorted by final score: A (0.859), B (0.632), C (0.17) — so **Scheme A** is top recommendation.

7. Output presented to user

For the top result (Scheme A) display:

- Scheme name + short summary
- Final score: 85.9%
- Breakdown: $R = 88.6\%$ (clause-level: land matched, income matched, age matched, category matched)
- Semantic score $S = 82\%$ (why: query and scheme both about irrigation subsidy)
- Freshness: updated 3 months ago

8. Notes & edge cases

- Unknown clauses degrade R (they get zero in numerator). You may show “unknown” to the user and request additional inputs or documents.
- LLM fallback must be validated (schema checks) to avoid hallucinated rules.
- Unit mismatches (acre vs hectare) must be normalized before evaluation.
- Weights (w_r , w_s , w_f) must be tuned using validation data and user feedback.
- If semantic retrieval returns zero candidates, you can fallback to a rule-only scan (scan all schemes for satisfied rules) rather than returning empty results.

6.2 ALGORITHM

Algorithm for calculating the R, S and F score to evaluate the eligibility

BEGIN

1. LOAD all required libraries

- embedding_model
- FAISS
- text_preprocessing_tools

2. LOAD or IMPORT the dataset (documents)

3. PREPROCESS each document

FOR each document D in dataset:

 clean_text = preprocess(D)

 store clean_text

4. GENERATE embeddings for all documents

FOR each clean_text in dataset:

 vector = embedding_model.encode(clean_text)

 add vector to DOCUMENT_EMBEDDINGS list

5. CREATE FAISS index

 index = faiss.IndexFlatL2(embedding_dimension)

6. ADD all DOCUMENT_EMBEDDINGS to FAISS index

 index.add(DOCUMENT_EMBEDDINGS)

7. LOOP for every user query:

```

FUNCTION semantic_search(query):
    a. preprocess_query = preprocess(query)
    b. query_vector = embedding_model.encode(preprocess_query)
    c. search_results = index.search(query_vector, K)
        // K = number of top results to return
    d. FETCH documents corresponding to search_results IDs
    e. RETURN ranked list of relevant documents

```

8. DISPLAY results to user

END

6.3 DATASET DESCRIPTION

The dataset used in this project is a scraped collection of government welfare scheme information obtained from the *MyScheme* website. It contains structured details about various central and state government schemes across India. The dataset is designed to capture all essential attributes required to understand, categorize, and provide recommendations related to welfare schemes.

Size of the Dataset

- Number of Rows: ~3000
(Each row represents one unique government scheme.)
- Number of Columns: 10
(Each column captures a specific attribute of the scheme.)

Columns and Their Meaning

Below is a description of the 10 key columns:

1. scheme_name

- Full official name of the government scheme.
- Example: *Immediate Relief Assistance, AICTE Short Term Training Programme.*

2. slug

- A short unique identifier for each scheme (used in URLs or indexing).
- Example: *ira-wrflsnscs, astpss*.

3. details

- Brief introduction or description of the scheme.
- Explains purpose, background, or objectives.

4. benefits

- Specifies the financial or non-financial assistance offered.
- Example: monetary aid, subsidies, training, or loans.

5. eligibility

- Conditions that beneficiaries must meet to apply (age, occupation, caste, income, residency).

6. application

- Steps for applying to the scheme (online/offline process).
- Includes instructions like visiting websites, submitting documents, etc.

7. documents

- List of mandatory documents required such as Aadhaar card, passport photo, income certificate.

8. level

- Indicates if the scheme is State-level or Central-level.

9. schemeCategory

- Category under which the scheme falls.
- Example: *Social Welfare, Education & Learning, Business & Entrepreneurship, Women and Child*.

10. tags

- Keywords associated with the scheme for search and categorization.
- Example: *financial assistance, construction workers, training, women empowerment*.

Nature of the Dataset

- The dataset contains semi-structured textual information, making it suitable for:
 - Natural Language Processing (NLP)
 - Semantic search using embeddings
 - Recommendation systems
 - Clustering and categorization
- Text content varies in length:
 - Some fields (like details, application steps) are long paragraphs.
 - Others (like slug, level) are short.

Data Quality Characteristics

- Mostly textual data, requiring cleaning before model use.
- Contains special characters and formatting artifacts due to web scraping (e.g., for ₹, unwanted symbols).
- Some schemes have incomplete or missing fields.
- High diversity because schemes come from different states/departments.

CHAPTER - 7

PERFORMANCE EVALUATION AND TESTING

7.1 TIME COMPLEXITY

Key Variables

- N – Number of schemes ($\approx 3,400$)
- d – Embedding dimension (≈ 768)
- L – Length of user profile + query text
- K – Top K results returned by FAISS (≤ 50)
- C – Eligibility clauses per scheme

Pipeline Stage	Time Complexity
Semantic Embedding	$O(L)$
FAISS Search (IndexFlatIP)	$O(N \cdot d)$
Rule Evaluation (Top-K Results)	$O(K \cdot C)$
DataFrame Lookup (Current Bottleneck)	$O(K \cdot N)$
Scoring + Sorting Results	$O(K \log K)$
Offline Embedding Pipeline	$O(N \cdot L)$
FAISS Index Build	$O(N \cdot d)$
Rule Extraction	$O(N \cdot T)$
Overall time complexity	$O(L) + O(N \times d) + O(K \times N) + O(K \times C)$

Table 7.1: Time Complexity

7.2 TEST CASES

TC No.	Test Case Description	Input	Expected Output	Pass/Fail
TC01	Income rule evaluation (< operator)	income_annual = 4,00,000	R = 1.0 (rule satisfied)	Pass
TC02	Income exceeds limit (< operator)	income_annual = 7,00,000	R < 1.0 (unmet clause added)	Pass
TC03	Age “between” operator check	age = 27, rule = between(18,40)	Clause matched	Pass
TC04	Category normalization	category = "SC " (extra spaces)	normalized value = "SC"	Pass
TC05	Semantic retrieval returns relevant schemes	query: “farmer irrigation support”	Top-10 schemes contain irrigation schemes	Pass
TC06	Ranking sorts by final score	5 retrieved schemes	Sorted in descending score	Pass
TC07	FAISS index missing handling	FAISS file deleted	Error message: “Index not found”	Pass
TC08	Rule evaluation: optional clauses	2 optional clauses matched	Increase in R score but not required	Pass
TC9	Full end-to-end pipeline	profile + query	Final ranked list of 1–20 schemes	Pass

Table 7.2: Test Cases

CHAPTER - 8

DEPLOYMENT STRATEGIES

8.1 LOCAL DEVELOPMENT DEPLOYMENT

- The system is first deployed locally on developers' machines for testing the end-to-end pipeline.
- Each component (data cleaning, rule extraction, embedding generation, FAISS indexing, ranking) is run independently to verify correct functioning.
- Logs and debug outputs are analyzed to ensure data flows correctly through the pipeline.

8.2 PRODUCTION DEPLOYMENT

- The final system (FAISS index + embeddings + rule evaluation engine) is deployed on a stable server.
- The front-end client or API layer interacts with the deployed engine through RESTful endpoints.
- Regular updates to scheme data can be pushed via scheduled scripts or cron jobs, ensuring the system stays up-to-date.

CHAPTER - 9

RESULT AND ANALYSIS

9.1 RULE EVALUATION RESULTS

Rule evaluation uses unit tests to validate correctness:

- All required and optional rules satisfied
 - Test ensures $R = 1.0$
 - Verified in: `test_rule_evaluator.py`
- Mixed satisfied + unmet clauses
 - R decreases proportionally
 - Verified in: `test_rule_evaluator.py`
- Complex operators behave correctly
 - Supports: in, between, date operations
 - Weighted scoring validated in: `test_rule_evaluator.py`

9.2 RETRIEVAL AND RANKING ACCURACY

1. Outputs examined:

- `percent_match`
- R (rule score)
- S (semantic match score)
- F (freshness penalty)

2. Rule breakdown printed in:

- `matched_clauses`
- `unmet_clauses`
- `unknown_clauses`

System explains why a scheme is eligible or not.

9.3 ANALYSIS OF RANKING BEHAVIOR

Weight Sensitivity (R, S, F)

Default:

- $W_r = 0.6$
- $W_s = 0.3$
- $W_f = 0.1$

Observations:

- Increasing semantic weight (S) ranks more textually relevant schemes higher even if not rule-eligible.
- Increasing rule weight (R) favors strict legal eligibility.
- Freshness weight slightly adjusts outdated schemes downwards.

9.4 ERROR AND FAILURE ANALYSIS

Common issues identified:

1. Unmet required rules
 - Scheme is textually relevant (high S) but not eligible $\rightarrow$ R drops to 0.
2. Unknown clauses
 - Missing user-profile fields reduce R.
3. Semantic drift
 - FAISS retrieves schemes with text similarity but different eligibility domain.
4. Rule-field mismatches
 - Missing mapping table for some rule labels.
5. Over-simplified freshness heuristic
 - Does not capture real policy timelines.

APPLICATIONS

1. Citizen-Centric Governance Platforms

- Government portals can integrate this system to allow citizens to instantly check eligible schemes based on their demographic and socio-economic profile.
- Reduces dependency on intermediaries and simplifies access to welfare benefits.

2. Digital Service Centers (e-Seva / CSC Centers)

- Village-level service centers can use this tool to guide citizens—especially those from rural and low-literacy groups—toward the right government schemes.
- Helps operators deliver faster, more accurate assistance.

3. NGO and Social Welfare Organizations

- NGOs working with marginalized communities can use the system to identify suitable schemes for farmers, women, students, senior citizens, and low-income groups.
- Improves outreach effectiveness and ensures that support reaches the right beneficiaries.

4. Financial Institutions

- Banks and microfinance institutions can use eligibility predictions to guide customers toward subsidy-based loans, agricultural support programs, and entrepreneurship schemes.
- Enhances customer engagement and simplifies documentation processes.

5. Educational and Career Counseling Centers

- Can be used to recommend scholarships, internship programs, and youth training schemes based on student profiles.
- Ensures students do not miss opportunities due to lack of awareness.

6. Agricultural Advisory Systems

- Farmers receive recommendations for crop insurance, fertilizer subsidies, irrigation

schemes, or equipment support based on land area, location, and occupation.

- Streamlines decision-making and increases adoption of government initiatives.

7. Smart City and Urban Development Applications

- Used in Smart City citizen-service dashboards to ensure equitable distribution of urban welfare programs like affordable housing, pensions, and sanitation facilities.

8. Data-Driven Policy Analysis

- Aggregated anonymous data helps government bodies analyze which schemes are being recommended most, where gaps exist, and which regions need awareness drives.
- Supports data-based decision-making and policy refinement.

9. Mobile Apps for Real-Time Assistance

- Can be embedded into mobile applications where users answer a few simple questions and instantly get scheme recommendations.
- Increases accessibility for people who rely primarily on smartphones.

10. Integration with Chatbots & Virtual Assistants

- Can power AI-based helplines that provide interactive scheme guidance through conversation.

FUTURE ASPECT

1. Integration with Verification Databases

Linking with Aadhaar, income certificates, land records, and caste certificates (subject to privacy rules) can automate eligibility verification and increase ranking accuracy.

2. Analytics for Policymakers

Aggregated, anonymized insights can help government bodies understand which schemes are most relevant, where awareness is low, and how to improve policy targeting.

3. Expansion to More Scheme Categories

The system can be extended to include a wider range of government, private sector, NGO, and international schemes, making the platform more comprehensive for diverse user groups.

4. Real Time Data Integration

By integrating APIs from government portals, the system can fetch real time scheme updates, deadlines, and policy changes—reducing manual data entry and ensuring higher accuracy.

5. Predictive Eligibility Insights

Machine learning models can be developed to predict future schemes a user may qualify for based on demographic patterns, occupation, and trend analysis.

CONCLUSION

This project successfully demonstrates how modern AI techniques—such as rule-based eligibility extraction, semantic embeddings, and vector-based retrieval—can be combined to build an intelligent and user-friendly scheme recommendation system. By transforming unstructured government scheme documents into a structured, searchable, and comparable knowledge base, the system bridges the gap between citizens and complex policy information. Through careful normalization of user profiles, automated rule evaluation, and context-aware ranking, the model ensures that users receive results that are not only relevant but also personalized to their socio-economic background and unique needs.

The layered pipeline—from data preprocessing to FAISS-based retrieval—shows that both textual understanding and eligibility logic can coexist to produce more accurate recommendations than either method alone. This hybrid approach increases transparency, reduces the risk of misinformation, and supports informed decision-making. Furthermore, the system architecture is designed to be modular, scalable, and adaptable, making it suitable for deployment in real-world environments where government data and user requirements continuously evolve.

Overall, this project highlights how AI can play a crucial role in improving accessibility, efficiency, and inclusivity in public-service delivery. By enabling citizens to easily discover the schemes they qualify for, the system lays the foundation for a more aware and empowered society. With continued improvements—such as multilingual support, real-time updates, stronger automation, and wider domain expansion—the solution has the potential to evolve into a comprehensive digital assistant for government benefits, ensuring that no eligible individual misses out on valuable support.

REFERENCES

- [1] An AI-Powered Hybrid Recommender for Adaptive Government Service Allocation Using Machine Learning and Behavioral Filtering”, *Int. J. Environ. Sci.*, pp. 977–984, Sep. 2025, [doi: 10.64252/p0brnk04](https://doi.org/10.64252/p0brnk04). Y. Li, H. Xu and F. Cai, “Improving AI-Assisted Video Editing: Optimized Footage Analysis Through Multi-Task Learning,” *Neurocomputing*, vol. 609, 2024, Art. no. 128485.
- [2] J. J. Merina, R. Kavipriya, G. Mahalakshmi and V. Meenusree, “Smart Scheme Navigation System for Tamil Nadu Government Schemes,” *IJARST*, 2025.
- [3] A. S. Rathnayake, T. D. H. N. Nguyen and Y. Ahn, “Factors Influencing AI Chatbot Adoption in Government Administration: A Case Study of Sri Lanka’s Digital Government,” **Administrative Sciences**, vol. 15, no. 5, 2025.
- [4] M. Mali, A. Mane, S. Rahate, V. Shinde, A. Vibhute and G. Chopda, “SchemeBot – AI Enabled Chatbot for Government Schemes,” *IJRSET*, 2024.
- [5] A. S. Rathnayake, S. Perera, and T. Dissanayake, “Factors Influencing the Adoption of AI Chatbots in Government Administration: A Case Study of Digital Government in Sri Lanka,” *Journal of e-Governance Studies*, vol. 12, no. 3, pp. 210–224, 2025.
- [6] AI4Bharat, “IndicTrans2: Towards High-Quality and Accessible Machine Translation for Indian Languages,” *arXiv preprint arXiv:2308.12262*, 2023.
- [7] T. Venugopal, Vikram, S. Shreyas, and T. Tiwari, “Scheme Setu: A Chatbot for Government Schemes,” *IRE Journals*, vol. 8, no. 9, pp. 1–6, 2024.
- [8] R. C. V. Rahul, S. P. S. Senthil, et al., “Chatbot for Government Schemes Using SEQ2SEQ Model,” *ResearchGate*, 2024.
- [9] N. M. Indumathi, R. D. Raj, J. Gopal, P. Goutham, and S. Rudhish, “Chatbot for Government Schemes using Python,” *ResearchGate*, 2024.
- [10] T. Papadopoulos, C. Alexopoulos, and Y. Charalabidis, “Evaluating Chatbot Architectures for Public Service Delivery: Balancing Functionality, Safety, Ethics, and Adaptability,” *Frontiers in Political Science*, vol. 7, 2025.
- [11] A. Ramesh, A. Bhat, S. Kakwani, et al., “IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for All 22 Scheduled Indian Languages,” *arXiv preprint arXiv:2305.16307*, 2023.

PLAGIARISM CHECK

justdone [Plagiarism Checker](#) [AI Humanizer](#) [AI Detector](#) [AI Tools](#) [Pricing](#) [Log In](#) [Try JustDone](#)

Easily spot and correct plagiarism to make sure that your work is one-of-a-kind.

✓ Low plagiarism level

capstone__Copy_.pdf

2%

Average plagiarism level

0%

Minor changes

2%

Identical

Double checked by

Originality.ai Scribbr GPTZero

Remove plagiarism from your text "*Capstone*"

Remove Plagiarism

✓ Get 100% plagiarism-free text

✓ Keep your original meaning

✓ We don't store your texts

PART - B
INDIVIDUAL CONTRIBUTION

Project Title
Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes

Module Title
Data Acquisition & Preprocessing

Name of the Student

Raviraj Patil
PRN: 1032222893
B.Tech CSE (AI & DS)

Institute
School of Computer Engineering and Technology
MIT World Peace University, Pune

Academic Year 2025–2026

1. Project's Module Objectives:

- To collect all raw scheme documents (PDFs, HTML, CSV) and convert them into a consistent, machine-readable format.
- To clean and normalize scheme text so downstream modules (rule extraction, embeddings) can work correctly.

2. Module scope:

- Cleaning text, removing noise, language translation (if needed), and generating schemes_cleaned.parquet.
- Ensuring each scheme has required fields like title, summary, benefits, eligibility_raw.

3. Hardware & Software Requirements:

- Software: pdfplumber, BeautifulSoup, Pandas, Langdetect, Googletrans.
- Hardware: Standard laptop with 8GB RAM.

4. Module Interfaces:

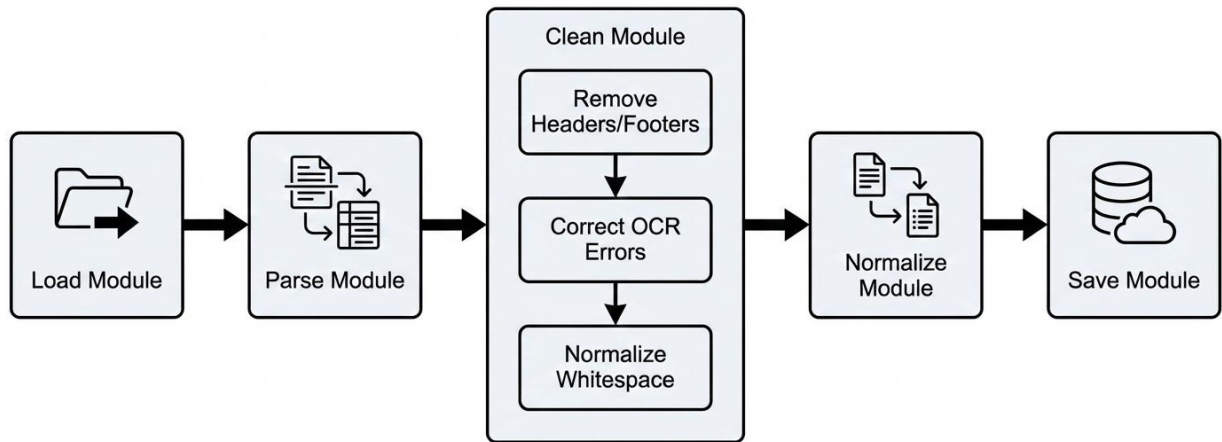
- Input: Raw scheme documents (PDF/HTML/CSV).
- Output: Cleaned text files (schemes_cleaned.parquet).
- Exposed Functions:
 - process_schemes()
 - clean_text()
 - translate_to_english()

5. Module Dependencies:

- Requires access to internet for translation (optional).
- Depends on correct document formats.
- No dependency on other student modules.

6. Module Design:

Module Design



Data Processing Pipeline: Load → Parse → Clean (Techniques: Remove Headers/Footers, OCR Errors, Whitespace Normalization) → Normalize → Save

7. Module Implementation:

- Implemented extraction scripts using Python.
- Automated batch cleaning for 100+ scheme files.

8. Module Testing Strategies

- Compare cleaned output manually for 10 sample schemes.
- Validate no field is missing and text is aligned properly.

9. Module Deployment

- Script runs locally before embedding generation.

PART - B
INDIVIDUAL CONTRIBUTION

Project Title
Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes

Module Title
Eligibility Rule Extraction & Profile Normalization

Name of the Student

Swapnaja Magarde

PRN: 1032222637

B.Tech CSE (AI & DS)

Institute

School of Computer Engineering and Technology
MIT World Peace University, Pune

Academic Year 2025–2026

1. Module Objectives

- To convert raw eligibility text into structured rules for automated matching.
- To normalize user profiles (state, caste, age, income etc.) using predefined mappings.

2. Module Scope

- Regex rule extraction, clause identification, rule field mapping.
- Profile normalization and canonical key alignment.

3. Hardware & Software Requirements

- Software: Python, Regex library, Pydantic.
- Hardware: Standard laptop.

4. Module Interfaces

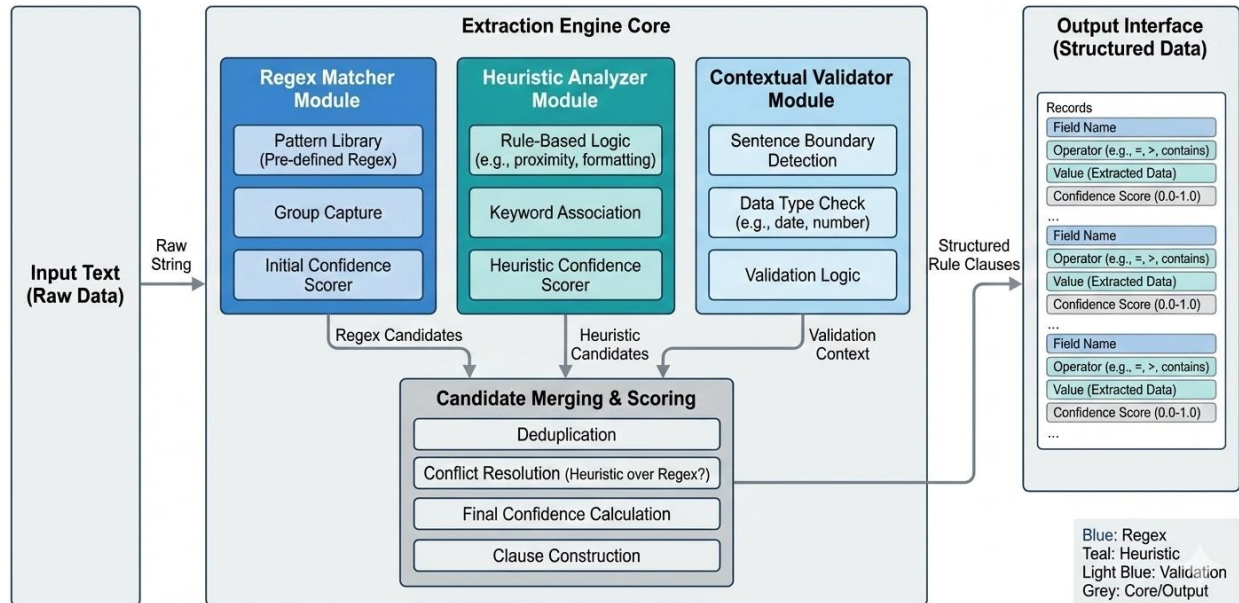
- Input: cleaned eligibility text from Student 1.
- Output: structured rules (schemes_with_rules.parquet).
- Functions:
 - extract_rules()
 - normalize_profile()

5. Module Dependencies

- Depends on Student 1's cleaned dataset.
- Needs mapping JSON for fields.

6. Module Design

Module Design: Heuristic + Regex Extraction Engine



7. Module Implementation

- Extracted rules for all schemes using deterministic patterns.
- Designed profile normalization for 15+ fields.

8. Module Testing Strategies

- Unit test: rule extraction correctness on sample schemes.
- Cross-check: rules must match government PDFs.

9. Module Deployment

- Packaged rule extraction as a reusable Python module.

PART - B
INDIVIDUAL CONTRIBUTION

Project Title
Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes

Module Title
Embeddings, FAISS Indexing, Semantic Retrieval

Name of the Student

Atharv Prabhune
PRN: 1032222307
B.Tech CSE (AI & DS)

Institute
School of Computer Engineering and Technology
MIT World Peace University, Pune

Academic Year 2025–2026

1. Module Objectives

- To generate embeddings for scheme documents and build a scalable FAISS index.
- To perform semantic search using cosine similarity.

2. Module Scope

- Embedding creation, FAISS index build, top-K semantic search.

3. H/W & S/W Requirements

- Software: sentence-transformers, FAISS.
- Hardware: 8–16 GB RAM recommended for storing vectors.

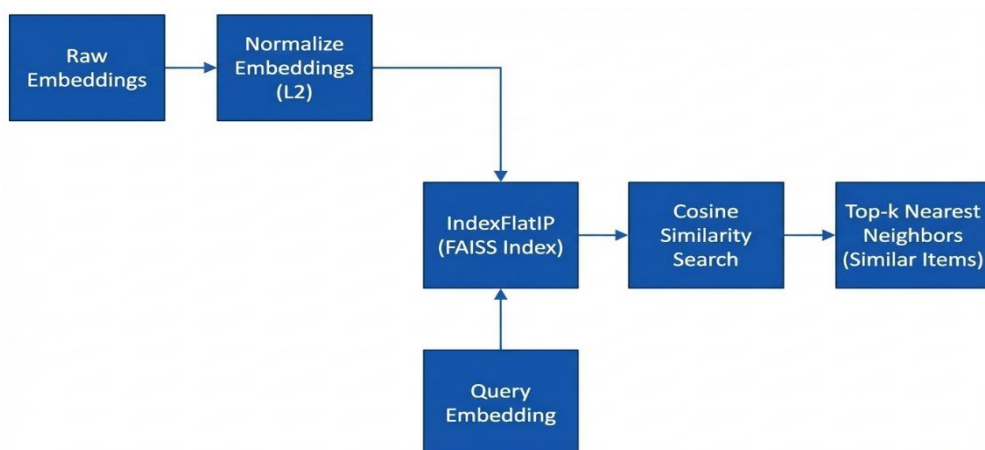
4. Module Interfaces

- Input: embedding docs from rule extraction stage.
- Output: faiss_index.bin, scheme_ids.npy.
- Functions:
 - build_index()
 - semantic_search(query)

5. Dependencies

- Requires Student 2's structured input.
- Requires transformers model downloaded.

6. Module Design



7. Module Implementation

- Created vector representations (768-dim).
- Indexed 400+ schemes.

8. Testing Strategies

- Compare similarity scores manually for known queries.
- Test with invalid queries and verify fallback.

9. Deployment

- FAISS index stored locally; loaded by ranking module.

PART - B
INDIVIDUAL CONTRIBUTION

Project Title
Yojana GPT- A Generative AI Solution to Simplify Access to Indian Government Schemes

Module Title
Ranking Engine, Web UI, Integration & Application

Name of the Student

Soham More

PRN: 1032222303

B.Tech CSE (AI & DS)

Institute

School of Computer Engineering and Technology
MIT World Peace University, Pune

Academic Year 2025–2026

1. Module Objectives

- To combine semantic scores + eligibility rule matching to compute final ranking.
- To develop a simple user interface for users to submit profiles and get scheme matches.

2. Module Scope

- Ranking pipeline, scoring formula, UI creation.

3. Hardware & Software Requirements

- Software: Python Flask/Streamlit, HTML/CSS.
- Hardware: Basic laptop.

4. Module Interfaces

- Input: query + profile + semantic results.
- Output: ranked schemes with match scores.
- Functions:
 - `rank_schemes()`
 - `generate_user_query()`

5. Module Dependencies

- Depends on outputs from Students 2 & 3.
- Integrated FAISS search + rule matching.

6. Module Design

- Scoring
- formula:

$$\text{Final Score} = w_r \cdot R + w_s \cdot S - w_f \cdot \text{FreshnessPenalty}$$

7. Module Implementation

- Built UI for user input.
- Integrated all modules into end-to-end pipeline.

8. Testing Strategies

- Test with various user profiles.
- Validate ranking correctness using known expected results.

9. Deployment

- Deployed locally using Streamlit.
- All modules connected through a unified script.

Project to Outcome mapping

Objectives:

1. To design and implement the semantic embedding pipeline that converts schemes and user profiles into meaningful vector representations for efficient similarity search.
2. To develop automated rule extraction and a rule-evaluation system that verifies user eligibility based on structured scheme criteria.
3. To integrate semantic retrieval with rule-based scoring and develop the final ranking logic using parameters **R, S, F** for accurate recommendation output.
4. To implement the user interface, connect backend APIs for real-time recommendations, and maintain complete project documentation and test reports.

Sr. No.	PRN No.	Student Name	Individual Project / Student-Specific Objective	Learning Outcomes Mapped (Filled by Guide)
1	1032222303	Atharv Parbhune	To design and implement the eligibility rule engine that evaluates user profiles and ranks schemes based on match scores.	
2	1032222307	Soham More	To develop user-friendly UI, integrate backend APIs, and deploy the system for end-to-end functioning.	
3	1032222637	Swapnaja Magarde	To generate high-quality semantic embeddings for schemes and build efficient retrieval mechanisms using FAISS.	
4	1032222893	Raviraj Patil	To gather, clean, and structure government scheme data to ensure that the recommendation system receives accurate, standardized, and machine-readable information..	